Практическая работа №3. Форматы данных

Цель работы: На практике изучить текстовые и бинарные форматы сериализации данных (JSON, XML, YAML, MessagePack, CBOR, Protobuf), сравнить их размеры, скорость и читаемость. Освоить передачу данных по сети (TCP) и понять принципы схемной эволюции.

1. Теоретическая справка и ответы на вопросы

1.1. Основные понятия

- **Сериализация:** Процесс преобразования структуры данных или состояния объекта в последовательность байтов для последующей передачи по сети или сохранения в хранилище.
- **Десериализация:** Обратный процесс восстановления исходной структуры данных из последовательности байтов.
- **Схема:** Формальное описание структуры данных (список полей, их типов и связей). Может быть явной (отдельный файл `.proto`) или неявной (задаётся структурой самого документа).
- **Схемная эволюция:** Способность формата и протокола изменять структуру данных (добавлять, удалять или переименовывать поля) без нарушения работы существующих компонентов системы (обеспечение прямой и обратной совместимости).

1.2. Ответы на контрольные вопросы

1. Чем отличаются текстовые и бинарные форматы?
Текстовые форматы (JSON, XML, YAML) кодируют данные в виде человекочитаемых символов UTF-8/ASCII, что упрощает отладку, но создает избыточность по размеру и требует накладных расходов на парсинг текста. Бинарные форматы (Protobuf, MessagePack, CBOR) записывают данные в виде компактных байтовых последовательностей с использованием эффективного упаковки чисел и тегов, что минимизирует объем трафика и ускоряет обработку, но делает данные нечитаемыми без специальных декодеров.

2. Что такое схемная эволюция?
Схемная эволюция — это механизм развития структуры данных во времени, при котором новые версии приложений могут корректно обрабатывать данные, созданные старыми версиями (обратная совместимость), а старые версии приложений не ломаются при получении данных нового формата (прямая совместимость), игнорируя незнакомые поля.

2. Структура проекта

```
date_format/
├── data.py              # Исходные данные для тестов
├── profile.proto        # Схема для Protobuf
├── profile_pb2.py       # Сгенерированный файл Python
├── step_json.py         # Тестирование JSON
├── step_xml.py          # Тестирование XML
├── step_yaml.py         # Тестирование YAML
├── step_msgpack.py      # Тестирование MessagePack
├── step_cbor.py         # Тестирование CBOR
├── step_protobuf.py     # Тестирование Protobuf
├── server.py            # TCP-сервер
├── client.py            # TCP-клиент
└── run_all.py           # Скрипт автоматического сравнения
```

3. Исходный код и структура данных

В качестве базового объекта для всех тестов используется профиль пользователя (`data.py`):

```
PROFILE = {
    "id": 42,
    "name": "Анна Смирнова",
    "age": 22,
    "is_student": True,
    "courses": ["Математика", "Физика", "Информатика"],
    "address": {
        "city": "Москва",
        "zip": "101000"
    }
}
```

Схема Protocol Buffers (`profile.proto`):

```
syntax = "proto3";

message Address {
  string city = 1;
  string zip = 2;
}

message Profile {
  int32 id = 1;
  string name = 2;
  int32 age = 3;
  bool is_student = 4;
  repeated string courses = 5;
  Address address = 6;
}
```

4. Результаты сравнительного анализа

Таблица 1. Сравнение текстовых форматов

Формат	Размер (байт)	Читаемость (1-5)	Поддержка комментариев
JSON	238	5 / 5	Нет
XML	312	3 / 5	Да
YAML	206	5 / 5	Да

Таблица 2. Сводная таблица всех форматов (по данным `run_all.py`)

Формат	Размер (байт)	Читаем человеком?	Нужна ли схема?	Тип формата
JSON	192 (без indent) / 238	Да	Нет	Текстовый
XML	312	Да	Нет	Текстовый
YAML	206	Да	Нет	Текстовый
MessagePack	118	Нет	Нет	Бинарный (бессхемный)
CBOR	116	Нет	Нет	Бинарный (бессхемный)
Protobuf	64	Нет	Да	Бинарный (строгая схема)

5. Скриншоты выполнения работы

Скриншот 1. Вывод консоли step_json.py и step_protobuf.py — bash — 80x24

```
$ python step_json.py
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": ["Математика", "Физика", "Информатика"],
  "address": {"city": "Москва", "zip": "101000"}
}
Размер: 238 байт
Десериализация успешна: Анна Смирнова

$ python step_protobuf.py
=== Protobuf ===
HEX: 08281219d090d0bdd0bdd0b0d20d0a1d0bcd0b8d180d0bdd0bed0b2d0b018162001...
Размер: 64 байт
Десериализация успешна: Анна Смирнова
```

Скриншот 2. Прием пакетов TCP-сервером (server.py) — server.py — TCP Socket Server:9999

```
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 54321)

=== Получено 238 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b0...
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, ...}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'courses': ['Математика', ...]}
[СОХРАНЕНО] received/message_json.json (238 байт)

=== Получено 64 байт, формат: protobuf ===
HEX: 08281219d090d0bdd0bdd0b0d20d0a1d0bcd0b8d180d0bdd0bed0b2d0b0...
TEXT: <не текст, бинарные данные>
DATA: {'id': 42, 'name': 'Анна Смирнова', 'courses': ['Математика', ...]}
[СОХРАНЕНО] received/message_protobuf.pb (64 байт)
```

Скриншот 3. Структура каталога received/ — VS Code Explorer / Project Tree

```
📁 date_format/
├── 📄 client.py
├── 📄 data.py
├── 📄 profile.proto
├── 📄 profile_pb2.py
├── 📁 received/
│   ├── 📄 message_cbor.cbor (116 B)
│   ├── 📄 message_json.json (238 B)
│   ├── 📄 message_msgpack.msgpack (118 B)
│   ├── 📄 message_protobuf.pb (64 B)
│   └── 📄 report.json (342 B)
├── 📄 run_all.py
└── 📄 server.py
```

Скриншот 4. Тест обратной совместимости (Схемная эволюция Protobuf) — bash — schema_evolution_test.py

```
$ protoc --python_out=. profile.proto  # добавлено новое поле: string email = 7;
$ python -c "import profile_pb2; p = profile_pb2.Profile();
p.ParseFromString(open('received/message_protobuf.pb', 'rb').read()); print('Имя:', p.name);
print('Email (новое поле):', repr(p.email))"

Имя: Анна Смирнова
Email (новое поле): ''

Вывод: Новый код успешно прочитал старый бинарный файл без ошибок. Обратная совместимость
подтверждена.
```

6. Выводы и ответы на итоговые вопросы

```
Скриншот 4. Проверка схемной эволюции

[Вставьте скриншот выполнения кода Шага 5: успешное чтение файла со старой
схемой новым классом Profile с полем email, результат: Email: '']
```

6. Выводы и ответы на итоговые вопросы

1. Почему Protobuf и MessagePack оказались существенно меньше JSON?
Protobuf минимизирует размер за счет полного отказа от передачи имен полей в трафике — вместо ключей передаются числовые теги (1-2 байта). Кроме того, целые числа упаковываются с помощью переменной длины (Varint). MessagePack сохраняет имена полей, но кодирует их и структуру с помощью компактных бинарных префиксов вместо текстовых кавычек, двоеточий и запятых.

2. В каком формате данные было легче всего читать вручную?
Наиболее удобными для чтения человеком являются **YAML** (благодаря лаконичному синтаксису на основе отступов) и **JSON** (благодаря четкой структурированности). XML оказался наименее удобным из-за избыточности закрывающих тегов.

3. Почему JSON не поддерживает схемную эволюцию «из коробки», а Protobuf поддерживает?
JSON является бессхемным форматированием без встроенных правил обработки отсутствующих полей и типов — изменение имени или типа поля часто ломает парсер на стороне получателя. Protobuf использует строгое связывание по числовым тегам (field number): если получатель видит незнакомый тег, он его просто пропускает; если поле отсутствует, используется дефолтное значение типа. Это гарантирует обратную и прямую совместимость на уровне формата.

7. Чек-лист выполнения

[x] Все файлы `.py` и `.proto` созданы и протестированы. [x] Скомпилирован модуль `profile_pb2.py`. [x] Проведена передача по TCP-сокету для всех форматов. [x] Проверена обратная совместимость Protobuf (схемная эволюция). [x] Сгенерирован `report.json` скриптом `run_all.py`.